

PostgreSQL, and the rows that do not change

A cluster of cooperating processes, an authentication file that decides who may connect from where, and MVCC shown directly — the same row before and after an UPDATE, with the transaction ids visible. Captured from a running lab.

Command `qlab run postgres-lab`

VM `postgres-lab`, 5432 on a dynamic host port

Credentials `labuser` / `labpass`

Outcome `qlab test postgres-lab` → 6 exercises, all passed

1. A cluster is several processes

```
psql (PostgreSQL) 14.24 (Ubuntu 14.24-0ubuntu0.22.04.1)
active
Ver Cluster Port Status Owner    Data directory          Log file
14  main    5432 online postgres /var/lib/postgresql/14/main /var/log/postgresql/postgresql-14-main.log
```

```
postgres      4832 /usr/lib/postgresql/14/bin/postgres -D /var/lib/postgresql/14/main -c config_file=/etc/
postgresql/14/main/postgresql.conf
postgres      4834 postgres: 14/main: checkpointer
postgres      4835 postgres: 14/main: background writer
postgres      4836 postgres: 14/main: walwriter
postgres      4837 postgres: 14/main: autovacuum launcher
postgres      4838 postgres: 14/main: stats collector
postgres      4839 postgres: 14/main: logical replication launcher
```

One supervisor and a set of specialists, each named after its job. They are worth knowing by name, because they are what you see in `ps` when something is wrong:

- **checkpointer** — flushes dirty pages and marks a point the database can recover from.
- **walwriter** — writes the write-ahead log. Every change is recorded there *before* it touches the data files, which is what makes crash recovery possible.
- **autovacuum launcher** — reclaims the dead row versions that section 4 is about. Left disabled, a busy table grows without bound.
- **background writer, stats collector, logical replication launcher.**

Postgres also calls a running instance a **cluster**: one server, one data directory, many databases. That word does not mean several machines here.

```
LISTEN 0.0.0.0:5432
LISTEN [::]:5432
```

```

              List of databases
  Name  | Owner  | Encoding | Collate | Ctype  | Access privileges
-----+-----+-----+-----+-----+-----
postgres | postgres | UTF8     | C.UTF-8 | C.UTF-8 |
template0 | postgres | UTF8     | C.UTF-8 | C.UTF-8 | =c/postgres      +
          |          |          |          |          | postgres=CTc/postgres
template1 | postgres | UTF8     | C.UTF-8 | C.UTF-8 | =c/postgres      +
          |          |          |          |          | postgres=CTc/postgres
testdb   | labuser | UTF8     | C.UTF-8 | C.UTF-8 | =Tc/labuser      +
          |          |          |          |          | labuser=CTc/labuser
(4 rows)
```

`template0` and `template1` are not clutter: a new database is a copy of a template. Anything you install into `template1` appears in every database created afterwards; `template0` is the pristine one you can always fall back to.

2. Who may connect, from where, and how

```
local all postgres peer
local all all peer
host all all 127.0.0.1/32 md5
host all all ::1/128 md5
local replication all peer
host replication all 127.0.0.1/32 md5
host replication all ::1/128 md5
host all all 0.0.0.0/0 md5
```

`pg_hba.conf` is read top to bottom and the **first matching line wins**. Each line is: connection type, database, user, address, method.

- `local ... peer` — over the unix socket, PostgreSQL asks the kernel which OS user you are and matches it to a role of the same name. No password exists. This is why `sudo -u postgres psql` works and `psql -U postgres` as another OS user does not.
- `host ... 127.0.0.1/32 md5` — over TCP, a password is required.

```
$ sudo -u postgres psql -c "SELECT current_user"    (unix socket, peer auth)
postgres|t

$ psql -h 127.0.0.1 -U labuser testdb              (TCP, password auth)
labuser|127.0.0.1
```

The same server, two connections, two identities — and `inet_server_addr()` returns NULL over the socket, which is a neat way to tell which path you took.

The last line of that file

`host all all 0.0.0.0/0 md5` accepts a password connection from any address on earth, to any database, as any role. It is here so you can point a GUI at the lab, and it is precisely the line to remove on anything real.

Role name	List of roles	Member of
	Attributes	
labuser	Create DB	{ }
postgres	Superuser, Create role, Create DB, Replication, Bypass RLS	{ }

Postgres has no separate notion of "user" and "group": both are **roles**, and a role that may log in is what other systems call a user. `postgres` is the superuser; `labuser` may only create databases.

3. A join, and what the planner did with it

```
$ SELECT u.name, COUNT(o.id) AS orders, SUM(o.amount) AS spent ...
 name | orders | spent
-----+-----+-----
 Alice |      2 | 1079.98
```

```

Charlie |      1 | 349.99
Bob      |      1 | 29.99
(3 rows)

$ EXPLAIN ANALYZE the same join
                                QUERY PLAN
-----
HashAggregate (cost=29.85..32.47 rows=210 width=154) (actual time=0.030..0.031 rows=3 loops=1)
  Group Key: u.id
  Batches: 1  Memory Usage: 40kB
  -> Hash Right Join (cost=14.72..28.40 rows=290 width=138) (actual time=0.020..0.023 rows=4 loops=1)
    Hash Cond: (o.user_id = u.id)
    -> Seq Scan on orders o (cost=0.00..12.90 rows=290 width=20) (actual time=0.001..0.002 rows=4
loops=1)
    -> Hash (cost=12.10..12.10 rows=210 width=122) (actual time=0.009..0.009 rows=3 loops=1)
      Buckets: 1024  Batches: 1  Memory Usage: 9kB

```

`EXPLAIN ANALYZE` does not estimate — it runs the query and reports what actually happened, including real timings and row counts. The gap between `rows=` estimated and `actual rows=` is the first thing to look at when a query is slow: if the planner's estimate is far off, it chose its strategy from bad information, and the fix is usually statistics rather than a different query.

4. MVCC, visible

```

Every row carries the transaction that created it and the one that deleted it.
xmin | xmax | id | name
-----+-----+-----+-----
737 | 739 | 1 | Alice
737 | 739 | 2 | Bob
737 | 739 | 3 | Charlie
(3 rows)

$ after an UPDATE, xmin changes: the old row version is still there until VACUUM
xmin | xmax | id | name
-----+-----+-----+-----
761 |    0 | 1 | Alice
737 | 739 | 2 | Bob
737 | 739 | 3 | Charlie
(3 rows)

```

This is PostgreSQL's central design decision, and here it is in the open.

Every row carries two hidden columns. `xmin` is the transaction that created this version of the row; `xmax` is the transaction that deleted or locked it, or `0` if none has.

Look at row 1 before and after the `UPDATE`. Its `xmin` changed. Postgres did not modify the row in place — it wrote a **new version** of it, and the old version is still on disk, now invisible to new transactions. That is why an `UPDATE` can be as expensive as an `INSERT`, and why readers never block writers: a reader simply keeps seeing the version that was current when it started.

The cost is that dead versions accumulate. Reclaiming them is exactly what `VACUUM`, and the autovacuum launcher in the process list above, exists to do.

5. Verification

```
Exercise 1: All 6 checks passed
Exercise 2: All 6 checks passed
Exercise 3: All 5 checks passed
Exercise 4: All 2 checks passed
Exercise 5: All 6 checks passed
Exercise 6: All 7 checks passed
Exercises run:      6
Exercises passed:   6
Exercises failed:   0
Exercises skipped:  0
All exercises passed!
```

6. What to take away

- A cluster is one server with many databases, not many machines.
- `pg_hba.conf` : first match wins. Order matters more than content.
- `peer` authenticates by OS identity over the socket; `md5` wants a password over TCP. Most "it works with sudo but not otherwise" confusion is this.
- Users and groups are both roles.
- `UPDATE` writes a new row version. Readers do not block writers, and `VACUUM` is what pays the bill.
- `EXPLAIN ANALYZE` runs the query. Compare estimated rows against actual.

`guide.md` in the plugin carries the exercises: queries, data manipulation, roles and privileges, backup with `pg_dump`, and configuration.